
Distributed Databases: Horizontal Partitioning

In this Lab, we will discover some notions related to *distributed databases (DDB)*. DDB refers to a database that is spread across multiple nodes, working together as a single database. To deal with this amount of data, the concept of *partitioning (Fragmentation)* is introduced. As a means to enhance the performance and reduce the total cost of storing large amounts of data.

① Useful Notions & Syntaxes

(a) Horizontal partitioning

- i) *Horizontal partitioning* in Oracle interests in dividing a table into *multiple subsets* based on *rows*. Each partition contains a *specific range of rows or meets certain criteria*. To perform horizontal partitioning, Oracle provides the *PARTITION BY* clause.
- ii) *Partition key* is a group of *one or more columns* that determines the partition in which each row in a partitioned table should proceed (e.g. **In table Principals, category='actor'**).
- iii) The horizontal partitioning has a set of *partitioning strategies*, those strategies are *Range, Interval, List, Reference, Hash, and Composite* partitioning:
 - *Range Partitioning*. Is a row partitioning based on ranges of values of the partitioning key. It is the most common type of partitioning and is often used with *dates*.

```
CREATE TABLE table_name
(
    column1 datatype,
    column2 datatype,
    -- additional columns
)
PARTITION BY RANGE (partitioning_column)
(
    PARTITION partition_name1 VALUES LESS THAN (value1),
    PARTITION partition_name2 VALUES LESS THAN (value2),
    -- additional partitions
    PARTITION partition_nameN VALUES LESS THAN (MAXVALUE)
);
```

- *Interval Partitioning*. is an extension of range partitioning. It is mainly applicable for scenarios where the database automatically manages the creation of partitions over time.

```

CREATE TABLE table_name
(
    column1 datatype,
    column2 datatype,
    -- additional columns
)
PARTITION BY RANGE (partitioning_column)
[INTERVAL (interval_value)]
(
    PARTITION partition_name1 VALUES LESS THAN (value1),
    PARTITION partition_name2 VALUES LESS THAN (value2),
    -- additional partitions
    PARTITION partition_nameN VALUES LESS THAN (MAXVALUE)
);

```

- *List partitioning*. Is the use of a list of separate values as the partition key for each partition. The partitioning key consists of one or more columns.

```

CREATE TABLE table_name
(
    column1 datatype,
    column2 datatype,
    -- additional columns
)
PARTITION BY LIST (partitioning_column)
(
    PARTITION partition_name1 VALUES ('value1', 'value2', ...),
    PARTITION partition_name2 VALUES ('value3', 'value4', ...),
    -- additional partitions
    PARTITION partition_nameN VALUES (DEFAULT)
);

```

- *Hash partitioning*. Is a partitioning way, where, the database designs rows into partitions based on a *hashing algorithm*, by applying the user-specified partitioning key. The resulting partitions are almost the same size.

```

CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    -- additional columns
)
PARTITION BY HASH (partitioning_column)
PARTITIONS X;---X is a Number

```

- *Reference partitioning*. Is a partitioning strategy for a *child table* is exclusively defined through the *foreign key relationship* with a *parent table*. For each partition in the parent table, strictly one equivalent partition exists in the child table. The parent table stores the parent records in a specific partition, and the child table stores the child records in the corresponding partition.

```

-- Creating a parent table
CREATE TABLE parent_table (
    parent_id NUMBER PRIMARY KEY,
    parent_name VARCHAR2(50)
);

-- Creating a child table with reference partitioning
CREATE TABLE child_table (
    child_id NUMBER PRIMARY KEY,
    child_name VARCHAR2(50),
    parent_id NUMBER,
    CONSTRAINT fk_parent FOREIGN KEY (parent_id) REFERENCES parent_table(parent_id)
)
PARTITION BY REFERENCE (fk_parent);

```

- After the creation of the fragmented tables is crucial to upload the data from the primary tables to the new ones using the Oracle command *INSERT INTO* the fragmented table the whole selection of attributes from the old table.

To check the correct creation of the new fragmented table, we can use the view:

```

-----Check the segment and
-----partition creation
SELECT
    table_name,
    partition_name,
    tablespace_name
FROM
    DBA_TAB_PARTITIONS
WHERE
    table_name = 'TABLE_NAME' AND
    table_owner = 'SCHEMA_Name';

-----
SELECT
    segment_type,
    tablespace_name,
    partition_name,
    bytes
FROM
    DBA_SEGMENTS
WHERE
    segment_name = 'TABLE_NAME' AND
    owner = 'SCHEMA_Name';

```

- *Single-level partitioning* operates with only one method of data distribution, e.g., list partitioning or only range partitioning. *In composite partitioning*, a table is partitioned by one data distribution method, and next each partition is further divided into subpartitions using a second data distribution method, e.g., Rang-Rang, List-List, Rang-Hash, Hash-Rang, ... Therefore, we can have several syntaxes for it, such as:

```

-----List-Hash
CREATE TABLE table_name
( column1 datatype,
  column2 datatype,
  -- additional columns
)
PARTITION BY LIST (partitioning_column1)
SUBPARTITION BY HASH (partitioning_column2)
SUBPARTITIONS X-- X is the number of subpartitions
( PARTITION partition_name1 VALUES ('Value1', 'Value2',...)
, PARTITION partition_name2 VALUES ('Value3', 'Value4',...)
-- additional partitions
, PARTITION partition_nameN VALUES (Default)
);

```

```

-----List-List
create table_name
( column1 datatype,
  column2 datatype,
  -- additional columns
)
PARTITION BY LIST (partitioning_column1)
SUBPARTITION BY LIST (partitioning_column2)
( PARTITION partition_name1 VALUES ('Value1', 'Value2',...)
  ( SUBPARTITION Subpartition_name11 VALUES ('Value11')
    , -- additional Subpartitions
  )
, PARTITION partition_name2 VALUES ('Value3', 'Value4',...)
  ( SUBPARTITION Subpartition_name21 VALUES ('Value21')
    , -- additional Subpartitions
  )
);

```

- To check the subpartition creation (after the Insertion), the following view can be used:

```

----Check the subpartition creation
SELECT
SELECT
  table_owner,
  table_name,
  subpartition_name,
  tablespace_name
FROM
  DBA_TAB_SUBPARTITIONS
WHERE
  table_name = 'TABLE_NAME' AND
  table_owner = 'SCHEMA_NAME';

```

- If we want to rewrite the query after the fragmentation, we can use the following syntax:

```

SELECT * FROM Frag_Tab PARTITION (partition_name);

```

- iv) *Partitions Manipulation.* After the creation of the partitions, a set of operations can be performed on them, regarding their needs. The table below summarizes some of the common types of manipulations that can be done on a partition:

Option	SQLPlus	Description
Add a New Partition	ADD PARTITION	Adds a new partition to an existing partitioned table
Split an Existing Partition	SPLIT PARTITION	Splits an existing partition into two partitions at a specified boundary value
Merge Two Partitions	MERGE PARTITIONS	Merges two adjacent partitions into a single partition
Drop a Partition	DROP PARTITION	Removes a partition and its data from the partitioned table.
Truncate a Partition	TRUNCATE PARTITION	Removes all rows from a partition but retains the partition structure
Renaming a Partition	RENAME PARTITION	Renames a partition in a partitioned table

- Some of the manipulation's operations can be like the following:

```
----Some Syntaxes
ALTER TABLE Table_name
RENAME PARTITION Old_partition_name
TO new_partition_name;

ALTER TABLE Table_name
ADD PARTITION partition_name
VALUES LESS THAN (your_partition_value); --This One for Range Partitioning

ALTER TABLE Table_name
MERGE PARTITIONS partition_name1, partition_name2
INTO PARTITION partition_name_new;

ALTER TABLE Table_name
DROP PARTITION partition_name;
```

② Lab Work

Important Remark

In this Lab, we are not supposed to delete the original tables after the creation of the fragmented ones.

Part I

- How to perform the fragmentation for the following queries. Check the creation of the partitions and the subpartitions.
- Re-write the queries according to the obtained fragments. **What do you notice?**
- Use the Oracle command to create the Plan trees for the same queries before and after the fragmentation. **What do you notice?**

Queries

- Select all the information from the table stuff where Deathyear =0.
- Create an other table to select all the information from the table stuff where birthyear is 1980, 1990, and 2000.
- Select all the information from the table show where showType is ["short","movie"], and ["tvMovie","tvEpisode"]. Create two even subpartition for each partition, based on IDshow. Check the creation.
- Perform an even partitioning into four partitions on the table show based IDshow, apply the same strategy for the table Rating. Check the operations.
- Select all the information from the table Rating only for AvrgRating <= 5 and >=6, and create three even subpartitions, based on the IDshow. Check the creation.
- Select all the information from the table Rating only for NumVote <=5000 and >=MaxValue, and create two subpartitions for each, based on the AvrgRating <=5 and < MaxValue. Check the creation.

Part II

- i) Perform the following operations on the previous queries:

Queries

- i) Rename the default partition in query (i) *Default_Partition*, and check the renaming.
- ii) Keep only the partition *Deathyear* = 0, of the query (i). Check the deletion.
- iii) Merge the first two partitions in query (ii) in one partition, and check the merging.
- iv) Drop the partitions with the smallest size within all the previous queries.

③ Homework

If you do not finish the queries in the lab session, you need to finish them at home as a homework.